

BERT-Based Sentiment Analysis Application

✓ Prerequisites

- Google Colab Account

Run below command to install dependencies

```
!pip install datasets
```

```
Collecting datasets
  Downloading datasets-3.1.0-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from datasets) (3.16.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from datasets) (1.26.4)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (17.0.0)
Collecting dill<0.3.9,>=0.3.0 (from datasets)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.10/dist-packages (from datasets) (2.32.3)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.10/dist-packages (from datasets) (4.66.6)
Collecting xxhash (from datasets)
  Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Collecting multiprocessing<0.70.17 (from datasets)
  Downloading multiprocessing-0.70.16-py310-none-any.whl.metadata (7.2 kB)
Collecting fsspec<2024.9.0,>=2023.1.0 (from fsspec[http]<=2024.9.0,>=2023.1.0->datasets)
  Downloading fsspec-2024.9.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from datasets) (3.10.10)
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (0.24.7)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from datasets) (24.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from datasets) (6.0.2)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (2.4.3)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.3.1)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (24.2.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (6.1.0)
Requirement already satisfied: yarl<2.0,>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.17.0)
Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (4.0.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub>=0.23.0->datasets) (4.12.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (2024.7.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.16.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.10/dist-packages (from yarl<2.0,>=1.12.0->aiohttp->dataset) (0.2.0)
Downloading datasets-3.1.0-py3-none-any.whl (480 kB)
480.6/480.6 kB 9.4 MB/s eta 0:00:00
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
116.3/116.3 kB 9.0 MB/s eta 0:00:00
Downloading fsspec-2024.9.0-py3-none-any.whl (179 kB)
179.3/179.3 kB 13.2 MB/s eta 0:00:00
Downloading multiprocessing-0.70.16-py310-none-any.whl (134 kB)
134.8/134.8 kB 10.4 MB/s eta 0:00:00
Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
194.1/194.1 kB 15.6 MB/s eta 0:00:00
Installing collected packages: xxhash, fsspec, dill, multiprocessing, datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2024.10.0
    Uninstalling fsspec-2024.10.0:
      Successfully uninstalled fsspec-2024.10.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the sou
gcsfs 2024.10.0 requires fsspec==2024.10.0, but you have fsspec 2024.9.0 which is incompatible.
Successfully installed datasets-3.1.0 dill-0.3.8 fsspec-2024.9.0 multiprocessing-0.70.16 xxhash-3.5.0
```

```
!pip install flask pyngrok transformers torch
```

```
Requirement already satisfied: flask in /usr/local/lib/python3.10/dist-packages (2.2.5)
Collecting pyngrok
  Downloading pyngrok-7.2.1-py3-none-any.whl.metadata (8.3 kB)
Requirement already satisfied: transformers in /usr/local/lib/python3.10/dist-packages (4.44.2)
Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages (2.5.0+cu121)
Requirement already satisfied: Werkzeug>=2.2.2 in /usr/local/lib/python3.10/dist-packages (from flask) (3.0.6)
Requirement already satisfied: Jinja2>=3.0 in /usr/local/lib/python3.10/dist-packages (from flask) (3.1.4)
Requirement already satisfied: itsdangerous>=2.0 in /usr/local/lib/python3.10/dist-packages (from flask) (2.2.0)
Requirement already satisfied: click>=8.0 in /usr/local/lib/python3.10/dist-packages (from flask) (8.1.7)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.10/dist-packages (from pyngrok) (6.0.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.16.1)
```

```
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.24.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.1)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2024.9.11)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.5)
Requirement already satisfied: tokenizers<0.20,>=0.19 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.19.1)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.6)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.10/dist-packages (from torch) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch) (3.4.2)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch) (2024.9.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2>=3.0->flask) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2024.8.30)
Downloading pyngrok-7.2.1-py3-none-any.whl (22 kB)
Installing collected packages: pyngrok
Successfully installed pyngrok-7.2.1
```

File Structure:

```
bert-sentiment-app/
├── app.py                # Flask application
├── sentiment_model.py    # BERT model logic for sentiment prediction
└── templates/
    ├── index.html        # Home page with text input
    └── result.html        # Displays sentiment results
```

File Structure:

```
bert-sentiment-app/ ├── app.py # Flask application ───
                    ├── sentiment_model.py # BERT model logic for sentiment prediction ───
                    └── templates/ ├── index.html # Home page with text input ───
                                └── result.html # Displays sentiment results
```

✓ Load Pre-trained Model

```
# @title Load Pre-trained Model
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
from datasets import load_dataset
```

Imports:

- > BertTokenizer: This is the tokenizer for BERT (Bidirectional Encoder Representations from Transformers). It is used to preprocess and convert text into a format that the BERT model can understand.
- > BertForSequenceClassification: This is the BERT model fine-tuned for sequence classification tasks. It can be used for tasks like sentiment analysis, text classification, etc.
- > Trainer: This is a class provided by the transformers library to facilitate the training and evaluation of models.
- > TrainingArguments: This class contains the arguments used to configure the training process.
- > load_dataset: This function from the datasets library is used to load datasets in a structured format.

```
# Load dataset
dataset = load_dataset('imdb')
```

Load dataset:

This line loads the IMDb dataset, a popular dataset for sentiment analysis, using the load_dataset function from the datasets library. The dataset is loaded into a variable named dataset.

```
# Load pre-trained tokenizer and model
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)
```

Loading pre-trained tokenizer

- > tokenizer = BertTokenizer.from_pretrained('bert-base-uncased'): This line loads a pre-trained BERT tokenizer. The bert-base-uncased model is used, which is a version of BERT that has been pre-trained on a large corpus of text and does not distinguish between uppercase and lowercase letters (uncased).

Loading pre-trained model -> `model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)`: This line loads a pre-trained BERT model specifically fine-tuned for sequence classification tasks. It uses the bert-base-uncased model and is configured to classify text into two categories (`num_labels=2`), which is typical for binary classification tasks like positive/negative sentiment analysis.

✓ Preparing smaller subsets of IMDB dataset

```
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
from datasets import load_dataset
```

Importing

- > BertTokenizer: A tokenizer for BERT that converts text into tokens.
- > BertForSequenceClassification: A pre-trained BERT model for sequence classification tasks.
- > Trainer and TrainingArguments: Utilities from the transformers library to facilitate model training.
- > load_dataset: A function to load datasets from the datasets library.

```
# Load a smaller subset of the dataset for faster training
dataset = load_dataset('imdb', split='train[:1%]')
test_dataset = load_dataset('imdb', split='test[:1%]')
```

- > dataset: Loads 1% of the training split of the IMDb dataset.
- > test_dataset: Loads 1% of the test split of the IMDb dataset.
- > This is done to speed up training and testing.

```
# Load pre-trained tokenizer and model
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)
```

Loads pre-trained model and tokenizer

- > tokenizer: Loads a pre-trained BERT tokenizer (bert-base-uncased).
- > model: Loads a pre-trained BERT model for sequence classification, configured for binary classification (`num_labels=2`).

```
# Tokenize dataset with shorter sequences
def tokenize_function(examples):
    return tokenizer(examples['text'], padding='max_length', truncation=True, max_length=128)
```

tokenize_function: A function that tokenizes the text in the dataset.

It: -> Pads the sequences to a fixed length (`max_length`).

- > Truncates sequences longer than the `max_length`.
- > Sets the maximum length of the sequences to 128 tokens.

```
tokenized_datasets = dataset.map(tokenize_function, batched=True)
tokenized_test_datasets = test_dataset.map(tokenize_function, batched=True)
```

Tokenizing dataset

- > tokenized_datasets: Applies the tokenize_function to the training dataset in batches.
- > tokenized_test_datasets: Applies the tokenize_function to the test dataset in batches.

```
tokenized_datasets = tokenized_datasets.remove_columns(['text'])
tokenized_test_datasets = tokenized_test_datasets.remove_columns(['text'])
```

Remove Text Column:

- > `tokenized_datasets.remove_columns(['text'])`: Removes the original text column from the tokenized training dataset.

-> `tokenized_test_datasets.remove_columns(['text'])`: Removes the original text column from the tokenized test dataset.

```
tokenized_datasets.set_format('torch')
tokenized_test_datasets.set_format('torch')
```

-> `tokenized_datasets.set_format('torch')`: Converts the tokenized training dataset to a format compatible with PyTorch.

-> `tokenized_test_datasets.set_format('torch')`: Converts the tokenized test dataset to a format compatible with PyTorch.

✓ Fine Tuning

```
import os
os.environ["WANDB_DISABLED"] = "true" # Disable Weights & Biases
```

Import os:

-> Imports the os module, which provides a way to interact with the operating system.

Disable Weights & Biases:

-> `os.environ["WANDB_DISABLED"] = "true"`: Sets an environment variable to disable Weights & Biases (a tool for tracking machine learning experiments). This is useful if you don't want to log your training runs to Weights & Biases

```
# Fine-tuning arguments with increased batch size and mixed precision
training_args = TrainingArguments(
    output_dir='./results',
    evaluation_strategy="epoch",
    per_device_train_batch_size=32, # Increase batch size
    per_device_eval_batch_size=32, # Increase batch size
    num_train_epochs=1,           # Reduce number of epochs
    weight_decay=0.01,
    fp16=True,                    # Use mixed precision training
    run_name='my_finetuning_experiment'
)
```

Fine-tuning Arguments:

-> `output_dir='./results'`: Specifies the directory where model checkpoints and logs will be saved.

-> `evaluation_strategy="epoch"`: Evaluates the model at the end of every epoch.

-> `per_device_train_batch_size=32`: Sets the batch size to 32 for training, which means 32 samples will be processed together in each training step.

-> `per_device_eval_batch_size=32`: Sets the batch size to 32 for evaluation.

-> `num_train_epochs=1`: Trains the model for 1 epoch, which is a complete pass over the training dataset.

-> `weight_decay=0.01`: Applies a weight decay regularization to prevent overfitting.

-> `fp16=True`: Uses mixed precision training, which speeds up the training process and reduces memory usage by using 16-bit floating point numbers where possible.

-> `run_name='my_finetuning_experiment'`: Names the training run, which is useful for logging and tracking purposes.

```
# Trainer setup
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets,
    eval_dataset=tokenized_test_datasets
)
```

-> `model=model`: Specifies the pre-trained model that will be fine-tuned.

-> `args=training_args`: Provides the training arguments defined earlier.

-> `train_dataset=tokenized_datasets`: Specifies the tokenized training dataset.

-> `eval_dataset=tokenized_test_datasets`: Specifies the tokenized test dataset.

```
# Fine-tune the model
trainer.train()
```

-> `trainer.train()`: Initiates the fine-tuning process of the model using the provided datasets and training arguments.

✓ Evaluation

```
# @title Evaluation
results = trainer.evaluate()
```

@title Evaluation:

-> This is a comment indicating that the following code block is for evaluating the model. The @title tag is often used in environments like Jupyter Notebooks or Colab to mark sections of the code.

`results = trainer.evaluate()`:

-> `trainer.evaluate()`: This function call evaluates the model on the evaluation dataset that was passed to the Trainer object during setup. It computes and returns various metrics, such as accuracy, loss, etc., based on the evaluation dataset.

`results`: The output of the evaluate function is stored in the variable `results`. This variable will contain a dictionary of evaluation metrics.

```
print(results)
```

this line prints the contents of the results dictionary to the console. It displays the evaluation metrics that were computed during the model evaluation step.

✓ Save the model

```
# @title Save the model
model.save_pretrained("./fine_tuned_model")
tokenizer.save_pretrained("./fine_tuned_model")
```

-> @title Save the model:

This is a comment indicating that the following code block is responsible for saving the fine-tuned model and tokenizer.

-> `model.save_pretrained("./fine_tuned_model")`:

`model`: This is the BERT model that has been fine-tuned.

`save_pretrained`: This method is used to save the model's configuration and weights to a directory. The method is part of the transformers library.

`./fine_tuned_model`: The directory where the model will be saved. If the directory doesn't exist, it will be created. This directory will contain files like `config.json` and `pytorch_model.bin` which store the model configuration and weights respectively.

-> `tokenizer.save_pretrained("./fine_tuned_model")`:

`tokenizer`: This is the BERT tokenizer that has been used for preprocessing the text data.

`save_pretrained`: This method saves the tokenizer's configuration and vocabulary to a directory. It ensures that the tokenizer can be reloaded in the same state as it was saved.

`./fine_tuned_model`: The same directory where the model was saved. This will include files like `vocab.txt`, `tokenizer_config.json`, and others that are necessary to reconstruct the tokenizer.

✓ Setting up a function for predicting the sentiment of input text

```
from transformers import BertTokenizer, BertForSequenceClassification
import torch
```

-> `BertTokenizer` and `BertForSequenceClassification`: Imported from the transformers library to handle text tokenization and sequence classification using BERT.

-> `torch`: Imported from the PyTorch library for tensor operations and model inference.

```
# Load the fine-tuned BERT model and tokenizer
model = BertForSequenceClassification.from_pretrained('./fine_tuned_model') # Path to your saved model
tokenizer = BertTokenizer.from_pretrained('./fine_tuned_model') # Path to your saved tokenizer
```

Loading the Model and Tokenizer:

-> model: Loads the fine-tuned BERT model from the specified directory (./fine_tuned_model). This model is configured for sequence classification.

-> tokenizer: Loads the tokenizer from the same directory. The tokenizer will preprocess the text input in the same way it was done during training.

```
def predict_sentiment(text):
```

Defines a function named predict_sentiment that takes a single argument text, which is the input text for sentiment analysis.

```
# Tokenize the input text
inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=128)
```

Tokenizing the Input Text:

-> tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=128): Tokenizes the input text.

-> return_tensors="pt": Returns the tokenized inputs as PyTorch tensors.

-> padding=True: Pads the sequences to the maximum length.

-> truncation=True: Truncates sequences that are longer than the max_length.

-> max_length=128: Sets the maximum length of the sequences to 128 tokens

```
# Perform prediction (without calculating gradients for efficiency)
with torch.no_grad():
    outputs = model(**inputs)
```

Performing Prediction:

-> with torch.no_grad(): Disables gradient calculation for efficiency since we are only performing inference, not training.

-> outputs = model(inputs)**: Passes the tokenized inputs through the model to obtain the outputs.

```
# Get the logits and predict the sentiment (0 = Negative, 1 = Positive)
logits = outputs.logits
prediction = torch.argmax(logits, dim=-1).item()
```

Processing the Model Outputs:

-> logits = outputs.logits: Extracts the logits (raw model outputs) from the model outputs.

-> prediction = torch.argmax(logits, dim=-1).item(): Determines the index of the maximum logit value along the last dimension (dim=-1).

Converts the result to a Python integer. This index represents the predicted class (0 for Negative, 1 for Positive).

```
# Return sentiment as a string
return 'Positive' if prediction == 1 else 'Negative'
```

Returns the sentiment label as a string. If the prediction is 1, it returns 'Positive'; otherwise, it returns 'Negative'.

✓ Making a local application accessible over the internet

```
from pyngrok import ngrok
```

This line imports the ngrok module from the pyngrok library, which allows you to create secure tunnels to your localhost, making your local web server accessible over the internet.

```
# Start ngrok tunnel with explicit protocol
public_url = ngrok.connect(5000, "http") # Specify "http" protocol explicitly
```

-> ngrok.connect(5000, "http"): This function call creates a tunnel that exposes your local server running on port 5000 to the internet. By specifying "http" as the protocol, you are explicitly telling ngrok to use the HTTP protocol for the connection.

-> public_url: The returned value is a public URL generated by ngrok, which can be used to access your local server from anywhere on the internet.

```
print("Access the app at:", public_url)
```

This line prints the public URL generated by ngrok to the console. You can use this URL to access your local server from any device with internet access.

The above local application will provide this URL

Access the app at: NgrokTunnel: <https://1090-34-30-169-71.ngrok-free.app> -> <http://localhost:5000>

```
!ngrok authtoken 2oeRHze3bXEIDGXTKRxc0IwOSVl_2NmQGhdn374Cp4JcrHBFB
```

!ngrok authtoken 2oeRHze3bXEIDGXTKRxc0IwOSVl_2NmQGhdn374Cp4JcrHBFB registers the provided authentication token with your ngrok client in the notebook environment. This allows you to establish secure tunnels to your localhost, making your local web server accessible over the internet.

It will provide this as the output: uthtoken saved to configuration file: [/root/.config/ngrok/ngrok.yml](#)

✓ Sets up a complete web application for sentiment analysis using Flask and a fine-tuned BERT mode

```
from flask import Flask, request, render_template_string
from pyngrok import ngrok
from transformers import BertTokenizer, BertForSequenceClassification
import torch
import threading
```

-> Flask: The main class for creating a Flask web application.

-> request: Used to handle HTTP requests, such as form submissions.

-> render_template_string: Renders HTML templates from strings.

-> ngrok: Used to create a secure tunnel to the localhost, making the Flask app accessible over the internet.

-> BertTokenizer and BertForSequenceClassification: Imported from the transformers library to handle text tokenization and sequence classification using BERT.

-> torch: PyTorch library for tensor operations and model inference.

-> threading: Python's threading module to run the Flask app in a separate thread.

```
# Load the fine-tuned model and tokenizer
model = BertForSequenceClassification.from_pretrained('./fine_tuned_model') # Path to your saved model
tokenizer = BertTokenizer.from_pretrained('./fine_tuned_model') # Path to your saved tokenizer
```

-> model: Loads the fine-tuned BERT model from the specified directory.

-> tokenizer: Loads the tokenizer from the same directory, ensuring that the tokenization process is consistent with the model.

```
def predict_sentiment(text):
    """ Function to predict sentiment using BERT """
    inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=128)
    with torch.no_grad():
        outputs = model(**inputs)

    logits = outputs.logits
```

```
prediction = torch.argmax(logits, dim=-1).item()

return 'Positive' if prediction == 1 else 'Negative'
```

-> predict_sentiment(text): A function that takes input text, tokenizes it, passes it through the model, and returns the sentiment prediction.

-> inputs = tokenizer(...): Tokenizes the input text and converts it into a format suitable for the BERT model.

-> with torch.no_grad(): Disables gradient calculation for efficiency since we are only performing inference.

-> outputs = model(...): Passes the tokenized inputs through the model to get predictions.

-> logits = outputs.logits: Extracts the raw model outputs (logits).

-> prediction = torch.argmax(logits, dim=-1).item(): Determines the predicted class (0 for Negative, 1 for Positive).

-> return 'Positive' if prediction == 1 else 'Negative': Returns the sentiment as a string.

```
# Set up Flask app
app = Flask(__name__)
```

app = Flask(name): Creates an instance of the Flask application.

```
# HTML template for the input form (index.html)
index_html = '''
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Text Sentiment Classification</title>
</head>
<body>
    <h1>Text Sentiment Analysis with BERT</h1>
    <form action="/predict" method="POST">
        <label for="text">Enter text for sentiment analysis:</label><br>
        <textarea name="text" id="text" rows="4" cols="50" required></textarea><br><br>
        <button type="submit">Analyze Sentiment</button>
    </form>
</body>
</html>
'''
```

index_html: A string containing the HTML template for the input form. This form allows users to input text for sentiment analysis.

```
# HTML template for the result page (result.html)
result_html = '''
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sentiment Analysis Result</title>
</head>
<body>
    <h1>Sentiment Analysis Result</h1>
    <p><strong>Text:</strong> {{ text }}</p>
    <p><strong>Prediction:</strong> {{ result }}</p>
    <a href="/">Try another text</a>
</body>
</html>
'''
```

result_html: A string containing the HTML template for the result page. This page displays the input text and its sentiment prediction.

```
# Route to render the form
@app.route('/')
def home():
    return render_template_string(index_html)
```

Route to Render the Form:

@app.route('/'): Defines the route for the home page.

def home(): A view function that renders the input form using the index_html template.

```
# Route to handle the prediction
@app.route('/predict', methods=['POST'])
def predict():
    text = request.form['text'] # Get text from the form
    result = predict_sentiment(text) # Get sentiment prediction
    return render_template_string(result_html, text=text, result=result)
```

Route to Handle the Prediction:

-> @app.route('/predict', methods=['POST']): Defines the route for handling form submissions.

-> def predict(): A view function that processes the form data.

-> text = request.form['text']: Retrieves the input text from the form.

-> result = predict_sentiment(text): Gets the sentiment prediction.

-> return render_template_string(result_html, text=text, result=result): Renders the result page using the result_html template, passing in the input text and prediction.

```
# Set up ngrok tunnel for Flask
public_url = ngrok.connect(5000, "http") # Specify "http" protocol
print("Access the app at:", public_url)
```

Set Up ngrok Tunnel:

public_url = ngrok.connect(5000, "http"): Creates a tunnel to the localhost on port 5000 using ngrok, making the Flask app publicly accessible.

print("Access the app at:", public_url): Prints the public URL generated by ngrok

```
# Run Flask app in a separate thread
def run_flask():
    app.run(port=5000)
```

def run_flask(): Defines a function to run the Flask app on port 5000

```
# Start Flask server
flask_thread = threading.Thread(target=run_flask)
flask_thread.start()
```

-> flask_thread = threading.Thread(target=run_flask): Creates a new thread to run the Flask app.

-> flask_thread.start(): Starts the new thread, running the Flask app concurrently with other processes.